# Práctica 1
# Tecnologías de Registro Distribuido y Blockchain (BC)

*Álvaro Fernández Carrillo*

**1. Análisis y definición del escenario..... 2**
1.1 Contexto y problemática..... 2
1.2 Justificación del uso de Blockchain pública..... 2
**2. Diseño..... 2**
2.1 Definición del caso de uso..... 2
2.2 Actores y Diagrama de Casos de Uso..... 3
2.3 Definición de Estructuras de Datos, Variables y Funciones..... 3
**3. Implementación y Optimización de Gas..... 3**
3.1 Código Fuente en Solidity..... 3
3.2 Técnicas de Optimización de Gas Aplicadas..... 3
**4. Pruebas y Validación en Remix IDE..... 4**
4.1 Secuencia de Pruebas Realizadas..... 4

# 1. Análisis y definición del escenario

## 1.1 Contexto y problemática

La verificación de titulaciones y certificaciones académicas enfrenta problemas de fraude o falsificación de documentos. El crear un sistema o herramienta que verifique que un documento es válido, original y que no haya sido modificado es una necesidad real para las instituciones.

## 1.2 Justificación del uso de Blockchain pública

El uso de una blockchain pública e inmutable como Ethereum o sus redes de segunda capa proporciona:

- **Inmutabilidad:** Una vez registrado el certificado por la institución autorizada, este no puede ser alterado ni eliminado retrospectivamente.

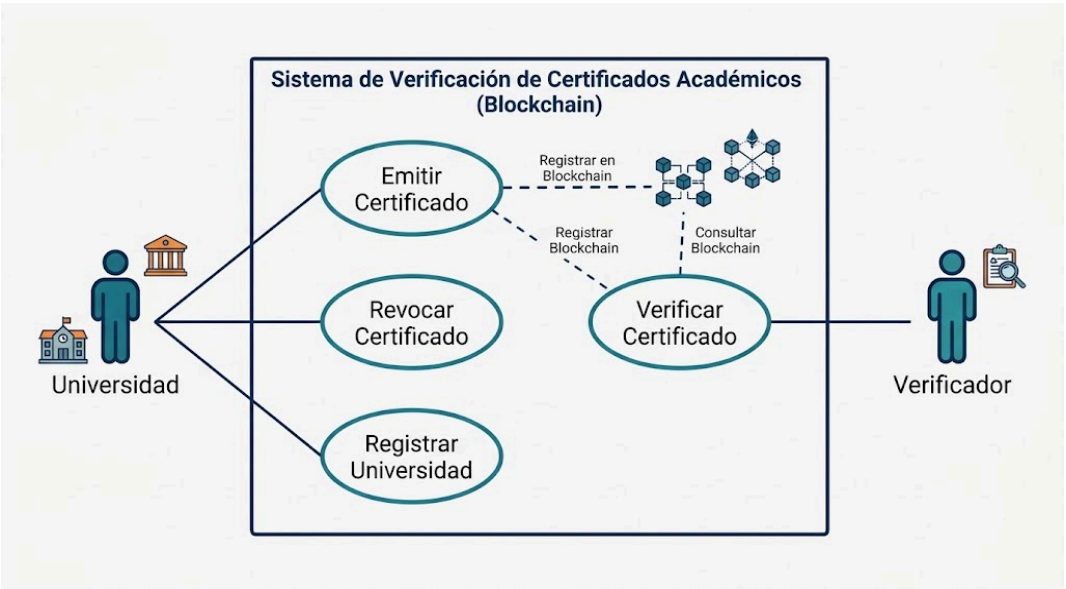
- **Trazabilidad y Autenticidad:** La identidad del emisor (**universityOwner**) queda sellada mediante firmas criptográficas en la red.

- **Descentralización y Disponibilidad:** No se requiere mantener un portal web o servidor expuesto 24/7 por parte de la universidad para verificar títulos. Cualquier tercero puede consultar directamente el estado del contrato inteligente en la cadena de bloques sin intermediarios ni costes adicionales.

# 2. Diseño

## 2.1 Definición del caso de uso

El sistema **AcademicCertificates** permite automatizar la emisión, verificación pública y eventual revocación de certificados académicos.



## 2.2 Actores y Diagrama de Casos de Uso

1. **Universidad (universityOwner):**
    - Despliega el contrato inteligente.
    - Emite nuevos certificados asociados a un identificador único hash (bytes32).
    - Revoca certificados previamente emitidos en caso de error administrativo o invalidez.

2. **Verificador Público (Empleador / Estudiante / Tercero):**
    - Consulta el estado e información de cualquier certificado mediante su identificador único de forma gratuita (función de solo lectura).

## 2.3 Definición de Estructuras de Datos, Variables y Funciones

- **Variables de estado:**
    - **universityOwner (address public immutable):** Almacena la dirección del creador/administrador del contrato.
    - **certificates (mapping(bytes32 => Certificate) public):** Tabla hash que asocia el identificador del certificado (bytes32) con su información estructurada.

- **Estructura (Certificate)**:
    - **studentName (string):** Nombre completo del alumno.
    - **degreeName (string):** Nombre de la titulación.
    - **issueDate (uint64):** Timestamp de la red en el momento de emisión.
    - **isValid (bool):** Estado de validez del título.

- **Eventos (Event):**
    - CertificateIssued(bytes32 indexed certId, string studentName, string degreeName)
    - CertificateRevoked(bytes32 indexed certId)

# 3. Implementación y Optimización de Gas

## 3.1 Código Fuente en Solidity

El código fuente se encuentra en **AcademicCertificates.sol**

## 3.2 Técnicas de Optimización de Gas Aplicadas

1. **Uso de la palabra clave immutable:**
   Al definir universityOwner como **immutable**, la dirección del propietario se escribe directamente dentro del bytecode del contrato durante el despliegue. Esto elimina las operaciones de lectura de almacenamiento permanente, ahorrando 2100 unidades de gas por cada llamada restringida.

2. **Uso de la ubicación de memoria calldata:**
   En las funciones externas (issueCertificate), los parámetros de cadena _studentName y _degreeName se declaran con **calldata**. Esto evita la copia innecesaria de datos de la

transacción a la memoria de ejecución (memory), optimizando el coste computacional.

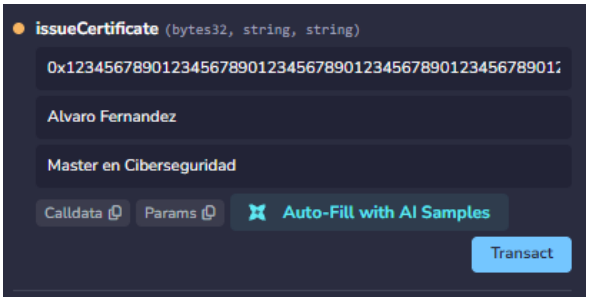
3. **Empaquetado de Variables dentro de la Estructura:**
   Al utilizar uint64 para la fecha (issueDate) en combinación con bool (isValid), Solidity empaqueta ambos tipos de datos en una única palabra de almacenamiento de 256 bits (32 bytes), ocupando un solo slot de almacenamiento permanente.

# 4. Pruebas y Validación en Remix IDE

## 4.1 Secuencia de Pruebas Realizadas

1. **Compilación y Despliegue:**
   - Entorno: Remix VM (Cancun)
   - El contrato es desplegado por la cuenta 0x… (asumiendo el rol de universityOwner)

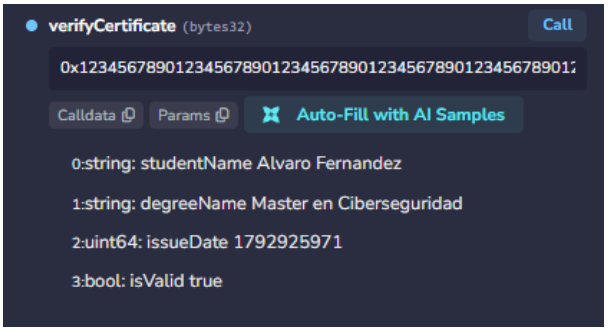
2. **Prueba de Emisión de Certificado (issueCertificate):**
   - Parámetro _certId: 0x1234567890123456789012345678901234567890123456789012345678901234
   - Parámetro _studentName: "Alvaro Fernandez"
   - Parámetro _degreeName: "Master en Ciberseguridad"
   - Resultado: Transacción ejecutada con éxito. Evento CertificateIssued emitido en los logs de la consola de Remix.



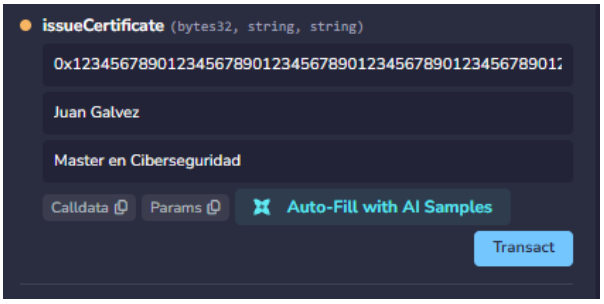


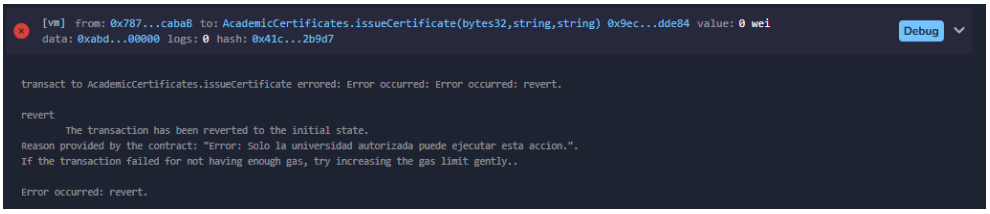


3. **Prueba de Verificación Pública (verifyCertificate):**
   - Se ejecuta la función pasando el hash 0x1111....
   - Resultado: Devuelve correctamente los datos correspondientes: "Alvaro Fernandez", "Master en Ciberseguridad", el timestamp actual y isValid = true.



4. **Prueba de Seguridad / Control de Acceso:**
    - Se cambia de cuenta en Remix a una cuenta secundaria (0xAb84...).
    - Se intenta ejecutar issueCertificate o revokeCertificate.
    - Resultado: La transacción falla y se revierte con el mensaje de error definido: "Error: Solo la universidad autorizada puede ejecutar esta accion.".




5. **Prueba de Revocación (revokeCertificate):**
    - Desde la cuenta universityOwner, se ejecuta revokeCertificate con el ID 0x1234567890123456789012345678901234567890123456789012345678901234
    - Se vuelve a consultar verifyCertificate.
    - Resultado: Devuelve isValid = false y emite el evento CertificateRevoked.

